MOTOSECURE | A NEXUS TECH LABS PRODUCT

Software Requirements Specification

Functional and non-functional requirements, business rules and acceptance criteria

Document ID	MS-SRS-001
Version	1.0 (draft for review)
Date	2 November 2026
Status	Draft for institutional review
Classification	Confidential
Publisher	NEXUS Tech Labs
Baseline	MotoSecure source archive as reviewed on 2 November 2026 (backend, web, mobile)
Related documents	MS-ARCH-001 Architecture; MS-SEC-001 Security, Privacy and Governance Policy; MS-ASR-001 Security Assessment Report
Approval	[Name, title and date - to be completed by the document owner]

This document describes the software as implemented in the reviewed source archive. Statements about hosting and deployment reflect the owner's reported configuration and were not independently verified. It is not a security certification, penetration-test report or compliance assessment.

Contents

1. Introduction 3
1.1 Purpose and scope 3
1.2 Conventions 3
2. Overall description 3
2.1 Product perspective 3
2.2 User classes 3
2.3 Operating environment and constraints 3
2.4 Assumptions and dependencies 4
3. Roles and permissions 4
4. Functional requirements 5
4.1 Authentication and account security 5
4.2 Municipalities (tenants) 5
4.3 Syndicates 6
4.4 Riders and badges 6
4.5 Motorcycles 7
4.6 Badge verification 7
4.7 Offline operation 7
4.8 Incident reporting 8
4.9 Audit logging 8
4.10 Dashboard and lists 9
4.11 Interface and localisation 9
5. Business rules 9
6. Non-functional requirements 10
7. Principal workflows 11
7.1 Onboarding a municipality and its first syndicate 11
7.2 Issuing a badge and checking it in the field 11
7.3 Recovering access 11
8. Functionality that is not implemented 11
9. Acceptance criteria 12
9.1 Pilot readiness gates 12
9.2 Acceptance of the verification feature 13

1. Introduction

1.1 Purpose and scope

This specification states what MotoSecure does and must do for the pilot deployment it is being prepared for. Requirements are derived from the implemented system: each one carries an implementation status and the evidence that supports it, so that reviewers can distinguish delivered behaviour from intended behaviour. Items that do not exist are listed as not implemented rather than omitted.

1.2 Conventions

Term	Meaning
Implemented	Present in the reviewed source and reachable through the product
Partially implemented	Present with a stated gap
Not implemented	Not present; listed so that expectations are explicit
Unit (executed)	Covered by an automated test that was run during the review and passed
Integration (written, not run)	Covered by an automated HTTP-level test that exists in the repository but could not be run in the review environment (no network access to install dependencies)
Reviewed only	Checked by reading the code; no automated test
Owner-tested manually	Exercised on a physical device by the owner; not recorded as an automated test

2. Overall description

2.1 Product perspective

MotoSecure is a multi-tenant web, mobile and API platform. A municipality (tenant) registers syndicates; syndicates enrol riders and motorcycles; the municipality approves them; riders carry a QR badge; enforcement officers verify badges on Android phones, with an offline fallback. A full description is in the System Architecture Document.

2.2 User classes

Role	Typical user	Interface
super_admin	Platform operator	Web (API access for municipality creation)
mayor	Mayor or senior council official	Web command dashboard
municipal_staff	Delegated council staff	Web command dashboard
syndicate_admin	Union president or secretary	Web portal (enrollment, own records)
enforcement_officer	Traffic or municipal police officer	Android app (web sign-in is technically possible)

2.3 Operating environment and constraints

- Browsers: current evergreen browsers (no browser matrix was tested). Android phones with a camera for scanning.
- Backend: Node.js 18 or later and MongoDB. Hosting and database providers as reported in the Architecture Document.

- Languages: English and French.
- Connectivity: officers may have intermittent coverage; scanning must degrade gracefully (FR-OFF).

2.4 Assumptions and dependencies

- TLS is provided by the hosting platform; the application does not itself terminate TLS.
- Email delivery for password reset depends on a configured Gmail SMTP account.
- Municipalities, syndicates and users are provisioned by the operator; there is no self-service user management (section 8).
- Legal and institutional requirements (data protection, hosting location, retention) are to be validated by the pilot partner and counsel (Governance Policy, section 11).

3. Roles and permissions

Permissions as implemented. "Own" means the caller's municipality; for a syndicate_admin it means the caller's own syndicate. Entries marked "not allowed" return HTTP 403 or 404.

Capability	super_admin	mayor	municipal_staff	syndicate_admin	enforcement_officer
Read rider and bike lists	All (or ?tenant=)	Own municipality	Own municipality	Own syndicate	Own municipality (no phone)
Read national ID / date of birth	Rider detail view	Rider detail view	Rider detail view	Own riders, detail view	**Not allowed**
Enrol riders and bikes	Any syndicate	Own municipality	Own municipality	Own syndicate, if approved	**Not allowed**
Edit rider and bike details	**Yes**	Own municipality	Own municipality	Own syndicate	**Not allowed**
Change rider status	**Yes**	**Yes**	**Yes**	**Not allowed**	**Not allowed**
Generate badge QR	**Yes**	**Yes**	**Yes**	Own syndicate	**Not allowed**
Read syndicates	All	Own municipality	Own municipality	Own syndicate only	Own municipality
Approve, reject, suspend syndicate	**Yes**	**Yes**	**Yes**	**Not allowed**	**Not allowed**
Edit syndicate contact details	**Yes**	**Yes**	**Yes**	Own syndicate	**Not allowed**
Create municipality	**Yes**	**Not allowed**	**Not allowed**	**Not allowed**	**Not allowed**
Edit municipality	All fields	Own contact fields	**Not allowed**	**Not allowed**	**Not allowed**
Verify badge: full view	Any rider	Own municipality	Own municipality	Limited view only	Own municipality
Record incident	**Yes**	Own municipality	Own municipality	**Not allowed**	Own municipality
Upload offline scans / snapshot	**Yes**	**Yes**	**Yes**	**Not allowed**	Own municipality
Read enforcement log and statistics	All	Own municipality	Own municipality	Own riders only	Own municipality
Read audit log	All	Own municipality	Own municipality	**Not allowed**	**Not allowed**

> **Observation for the pilot partner**
> An enforcement officer can read the syndicate list of their own municipality (including president contact details) and the rider and bike lists (without phone numbers). The mobile app does not need these; tightening them is recommended once web access for officers is confirmed to be unnecessary.

4. Functional requirements

4.1 Authentication and account security

ID	Requirement	Status	Verification
FR-AUTH-01	A user signs in with email and password and receives a signed token (HS256, default 7-day lifetime).	**Implemented**	Unit (executed): token verification rules Integration (written, not run): auth.test.js
FR-AUTH-02	Sign-in failure gives the same generic message for unknown accounts, wrong passwords and locked accounts.	**Implemented**	Integration (written, not run)
FR-AUTH-03	Five consecutive failed sign-ins lock the account for 15 minutes; the lock is audited.	**Implemented**	Integration (written, not run)
FR-AUTH-04	Every protected request re-validates the account (active) and rejects tokens issued before the last password change.	**Implemented**	Unit (executed)
FR-AUTH-05	Passwords on registration and reset have at least 8 characters, a letter and a digit, and at most 72 bytes.	**Implemented**	Unit (executed) Integration (written, not run)
FR-AUTH-06	A syndicate may self-register with a non-suspended municipality; this creates a pending syndicate and its administrator account.	**Implemented**	Integration (written, not run)
FR-AUTH-07	Password reset by emailed single-use link valid for one hour; the request response does not reveal whether the account exists; earlier sessions stop working after a reset.	**Implemented (delivery depends on email configuration)**	Unit (executed): session-age rule Integration (written, not run)
FR-AUTH-08	Signed-in password change, email verification at registration, multi-factor authentication, administrator-initiated reset.	**Not implemented**	-
FR-AUTH-09	Sign-out clears the client's token. There is no server-side token revocation list.	**Partially implemented**	Reviewed only

4.2 Municipalities (tenants)

ID	Requirement	Status	Verification
FR-TEN-01	A super_admin creates and edits municipalities; a mayor edits only their own municipality's contact details (not name, code, plan or status).	**Implemented**	Unit (executed): field allow-lists Integration (written, not run)
FR-TEN-02	The public enrollment list exposes only name, code and region of non-suspended municipalities.	**Implemented**	Integration (written, not run)

ID	Requirement	Status	Verification
FR-TEN-03	Screens for municipality onboarding and administration.	**Not implemented (API only)**	-
FR-TEN-04	Plan, billing, and enforcement of tenant settings (maxRidersPerSyndicate, requireBikeInsurance).	**Not implemented (values are stored, not enforced)**	-

4.3 Syndicates

ID	Requirement	Status	Verification
FR-SYN-01	Council users list, create and edit syndicates of their own municipality.	**Implemented**	Integration (written, not run)
FR-SYN-02	Council users approve, reject or suspend a syndicate; each change is audited with previous and new status.	**Implemented**	Integration (written, not run)
FR-SYN-03	A syndicate administrator sees and edits only their own syndicate's contact fields and cannot change status, trust score, name, member count or municipality.	**Implemented**	Unit (executed): allow-lists, scoping Integration (written, not run)
FR-SYN-04	Riders and bikes can be enrolled by a syndicate administrator only after the syndicate is approved.	**Implemented**	Integration (written, not run)
FR-SYN-05	Trust score derived from behaviour or incidents.	**Not implemented (manually entered number)**	-
FR-SYN-06	Member count maintained automatically.	**Not implemented**	-
FR-SYN-07	Upload and storage of syndicate documents.	**Not implemented (URL list field only)**	-

4.4 Riders and badges

ID	Requirement	Status	Verification
FR-RID-01	Riders are enrolled under a syndicate (by its approved administrator or by council). A new rider always starts under review and receives a server-generated badge id.	**Implemented (API; no web form)**	Integration (written, not run)
FR-RID-02	Only council users change a rider's status, through an audited route. Generic edits cannot change status, badge id, counters, municipality or syndicate.	**Implemented**	Unit (executed): allow-lists Integration (written, not run)
FR-RID-03	Badge ids are unique, immutable and unguessable in practice (48 random bits). A QR image can be generated by managers within their scope; officers cannot.	**Implemented**	Integration (written, not run)
FR-RID-04	National ID number and date of birth are excluded from all list queries and officer views; phone numbers are hidden from officers.	**Implemented**	Unit (executed): projections Integration (written, not run)
FR-RID-05	Photo capture, upload and storage.	**Not implemented (photo URL text field only)**	-

ID	Requirement	Status	Verification
FR-RID-06	Revocation takes effect at the next online verification, and on an offline device at its next snapshot refresh.	**Implemented**	Unit (executed) Integration (written, not run)

4.5 Motorcycles

ID	Requirement	Status	Verification
FR-BIK-01	Motorcycles are registered under a syndicate; plate numbers are unique within a municipality.	**Implemented**	Integration (written, not run)
FR-BIK-02	A bike can be linked only to a rider of the same municipality and syndicate; relinking updates both records.	**Implemented**	Integration (written, not run)
FR-BIK-03	Insurance or roadworthiness status affects the verification verdict.	**Not implemented (recorded and displayed only)**	-

4.6 Badge verification

ID	Requirement	Status	Verification
FR-VER-01	The verdict follows business rule BR-01.	**Implemented**	Unit (executed): verdict.test.js
FR-VER-02	Badge ids are trimmed and upper-cased; malformed ids are rejected with HTTP 400.	**Implemented**	Unit (executed) Integration (written, not run)
FR-VER-03	Anonymous callers receive a limited view (masked name and verdict), nothing is written, and they are limited to 30 requests per minute per client address.	**Implemented**	Unit (executed): limiter settings Integration (written, not run)
FR-VER-04	Officers and council users verifying a rider of their own municipality receive the full view; the scan is logged and the rider's count incremented.	**Implemented**	Unit (executed): view rules Integration (written, not run)
FR-VER-05	Officers of another municipality and syndicate administrators receive the limited view and nothing is written.	**Implemented**	Unit (executed) Integration (written, not run)
FR-VER-06	A supplied but invalid or expired token is rejected with HTTP 401 and is never treated as anonymous.	**Implemented**	Unit (executed): optional-auth behaviour Integration (written, not run)
FR-VER-07	Officers verify by camera QR scan or manual badge entry on Android.	**Implemented**	Owner-tested manually

4.7 Offline operation

ID	Requirement	Status	Verification
FR-OFF-01	The app caches a municipality snapshot while online.	**Implemented**	Reviewed only

ID	Requirement	Status	Verification
FR-OFF-02	On connectivity failure (no response, 12-second timeout, HTTP 502-504) the app answers from the snapshot, labels the result with the snapshot date, and queues the scan. Other errors are not answered from the cache.	**Implemented**	Reviewed only Unit (executed): verdict parity with server
FR-OFF-03	Queued scans upload in batches of at most 200, with unique scan ids, only for riders of the officer's municipality, with valid verdicts, no older than 30 days; future timestamps are clamped; duplicates are ignored.	**Implemented**	Integration (written, not run)
FR-OFF-04	A signed-in officer stays signed in across a cold start without network; all cached rider data and queued scans are removed on sign-out.	**Implemented**	Reviewed only
FR-OFF-05	Encrypted storage of the snapshot on the device.	**Not implemented**	-
FR-OFF-06	Incident reporting while offline.	**Not implemented (requires connectivity)**	-

4.8 Incident reporting

ID	Requirement	Status	Verification
FR-INC-01	Officers and council users record an incident, warning or impound against a rider of their own municipality, with severity (low to critical) and a description of at most 1,000 characters.	**Implemented**	Integration (written, not run)
FR-INC-02	Recording an incident increments the rider's incident count and appears in the enforcement log.	**Implemented**	Integration (written, not run)
FR-INC-03	Photo evidence, case workflow, fines or penalties.	**Not implemented**	-

4.9 Audit logging

ID	Requirement	Status	Verification
FR-AUD-01	Administrative actions are recorded with actor, role, action, target, before and after values, IP address and time (list in the Architecture Document, section 7.6).	**Implemented**	Integration (written, not run)
FR-AUD-02	The audit log is append-only through the application and readable only by council roles within their own municipality.	**Implemented (database-level immutability not enforced)**	Integration (written, not run)
FR-AUD-03	A failure to write an audit entry is logged to the server console and does not block the user's action.	**Implemented (design decision)**	Reviewed only
FR-AUD-04	Search, filter or export screens for the audit log.	**Not implemented**	-

4.10 Dashboard and lists

ID	Requirement	Status	Verification
FR-DASH-01	The dashboard shows riders, active riders, bikes, syndicates, pending approvals and incidents in the last 30 days, scoped to the caller (a syndicate administrator sees only their own syndicate).	**Implemented**	Integration (written, not run)
FR-DASH-02	Lists of syndicates, riders, bikes and the enforcement log, with tables that collapse to cards on small screens.	**Implemented**	Reviewed only
FR-DASH-03	Pagination, search and export.	**Not implemented (lists capped at 1,000 by default)**	-

4.11 Interface and localisation

ID	Requirement	Status	Verification
FR-UI-01	Web and mobile offer English and French.	**Partially implemented (some values, for example bike status, display untranslated)**	Reviewed only
FR-UI-02	Light and dark themes, remembered per device.	**Implemented**	Reviewed only
FR-UI-03	The web application is usable on phone-sized screens.	**Implemented**	Reviewed only
FR-UI-04	MotoSecure branding with a NEXUS Tech Labs credit.	**Implemented**	Reviewed only

5. Business rules

ID	Rule	State
BR-01	Verdict precedence: unknown badge is not_found; status suspended is suspended; status revoked or under_review is flagged; licence expiry in the past is expired; otherwise valid.	**Implemented; Unit (executed)**
BR-02	A new rider starts under_review. Only council users move a rider to active, suspended or revoked, and every change is audited.	**Implemented**
BR-03	A syndicate must be approved by the council before its administrator can enrol riders or bikes.	**Implemented**
BR-04	A syndicate administrator can see and change only their own syndicate's data.	**Implemented; Unit (executed)**
BR-05	All reads and writes are limited to the caller's municipality. Only super_admin can cross municipalities.	**Implemented; Unit (executed)**
BR-06	The detail returned by verification depends on the caller and the rider's municipality: full for officers and council within their own municipality, otherwise limited.	**Implemented; Unit (executed)**
BR-07	Badge ids are generated by the server, unique and cannot be changed.	**Implemented**
BR-08	Offline results are indicative: they show the snapshot date, and revocations after the last sync are not reflected until the device reconnects.	**Implemented**
BR-09	Five consecutive failed sign-ins lock an account for 15 minutes.	**Implemented**

ID	Rule	State
BR-10	A suspended or rejected syndicate does not currently change the verdicts of its riders.	**Decision required: should suspending a syndicate flag its riders?**
BR-11	Insurance and roadworthiness are recorded but do not affect the verdict.	**Decision required**
BR-12	Officers from one municipality verifying riders of another receive only a limited view.	**Decision required: confirm this matches inter-municipal enforcement practice**

6. Non-functional requirements

ID	Requirement	Status	Verification
NFR-SEC-01	Data in transit is protected by TLS.	**Depends on deployment (provided by hosting platform)**	Not testable from the repository
NFR-SEC-02	Passwords are stored as bcrypt hashes (cost 12 by default); reset tokens as SHA-256 hashes.	**Implemented**	Reviewed only
NFR-SEC-03	The API refuses to start without a database URI and a strong signing secret.	**Implemented**	Unit (executed): config.test.js
NFR-SEC-04	Rate limits: sign-in 30 per 15 minutes, password reset 5 per hour, registration 10 per hour, anonymous verification 30 per minute, global 300 per minute per client.	**Implemented**	Unit (executed): limiter settings Not covered: actual throttling (disabled in the test environment)
NFR-SEC-05	Security headers, CORS allow-list, 100 KB body limit, rejection of operator-style keys.	**Implemented**	Unit (executed): operator rejection Integration (written, not run): CORS, headers, body size
NFR-SEC-06	Production error responses do not reveal internals.	**Implemented**	Unit (executed): errorHandler
NFR-PRV-01	Personal data is minimised by role (section 3).	**Implemented**	Unit (executed) Integration (written, not run)
NFR-PRV-02	Defined retention periods and deletion or anonymisation procedures.	**Not implemented (no deletion routes; periods to be set by the data controller)**	-
NFR-PRV-03	Tooling to serve data-subject access, correction and erasure requests.	**Not implemented**	-
NFR-PERF-01	Response-time and capacity targets.	**Not defined; no performance or load testing performed**	-
NFR-AVAIL-01	Availability target and recovery objectives.	**Not defined; the reported hosting tier sleeps when idle**	-
NFR-OBS-01	Central logging, monitoring and alerting.	**Not implemented (request logs only)**	-

ID	Requirement	Status	Verification
NFR-BKP-01	Automated backups with tested restore.	**Not implemented in the repository**	-
NFR-MNT-01	Automated tests run on every change.	**Partially implemented: 51 of 51 unit tests executed and passing; 122 integration tests written, not run; no UI tests; no CI**	Test runner: backend/tests/run.js
NFR-I18N-01	English and French throughout.	**Partially implemented (see FR-UI-01)**	Reviewed only

7. Principal workflows

7.1 Onboarding a municipality and its first syndicate

1. The operator (super_admin) creates the municipality through the API.
2. A user account for the mayor and for each officer is created by the operator (there is no user-management screen; see section 8).
3. A syndicate representative registers through the public enrollment page and waits for approval.
4. The mayor or staff approve the syndicate; the syndicate administrator can then enrol riders and bikes (currently through the API).

7.2 Issuing a badge and checking it in the field

1. A rider is enrolled (status under_review); council reviews documents and activates the rider.
2. A manager generates the badge QR and prints it for the rider.
3. An officer scans the QR (or types the badge id); the app shows the verdict and, for a rider of the officer's municipality, the rider's name, syndicate, plate number and licence expiry for comparison with the person in front of them.
4. If the officer is offline the app answers from its snapshot and uploads the scan later.

7.3 Recovering access

1. The user requests a reset using their account email.
2. A one-hour link is emailed; opening it lets the user choose a new password that meets the policy.
3. All earlier sessions stop working and the account lock, if any, is cleared.

8. Functionality that is not implemented

Listed so that the pilot partner's expectations match the product. None of these is implied elsewhere in this specification.

Area	Gap	Impact on a pilot
User management	No API or screen to create officer, staff or mayor accounts, deactivate users, or let a user change their own password	Provisioning requires operator tooling or database access; this is the main operational prerequisite
Rider and bike enrolment screens	The API exists; the web app has list pages only	Syndicates cannot enrol members without a tool

Area	Gap	Impact on a pilot
Authentication strength	No multi-factor authentication, email verification or refresh tokens	Privileged accounts rely on password strength alone
Files	No photo or document upload	Badges cannot carry a stored photo
Verdict inputs	Insurance, roadworthiness and syndicate suspension do not affect the verdict	Decisions BR-10 and BR-11
Tenant settings	maxRidersPerSyndicate and requireBikeInsurance are stored, not enforced	Council policy cannot be configured
Counters	Syndicate member count and trust score are not computed	Figures shown for these are not derived data
Notifications	No SMS, email or push notifications (other than password reset)	Approvals are not announced
Data lifecycle	No retention, deletion or export functions	Needed to meet data-protection obligations once defined
Scale features	No pagination, search or reporting exports	Large municipalities will outgrow the lists

9. Acceptance criteria

9.1 Pilot readiness gates

A pilot should not start until each gate below is satisfied. The state column reports the position at the time of writing.

Gate	Criterion	Evidence required	State
G1	All unit tests pass	Test runner output	**Pass: 51 of 51 executed**
G2	Integration suite passes on a clean install against an isolated database	npm install, then npm test in backend/	**Not run (could not be executed in the review environment)**
G3	No high or critical advisories in production dependencies	npm audit --omit=dev in backend, web and mobile	**Not run (no network in the review environment)**
G4	Demonstration accounts and data removed from any pilot or production database	Database inspection	**Open**
G5	All credentials shared during development rotated; secrets held only in the hosting dashboards	Rotation record	**Open**
G6	Separate environments; automated backups with a tested restore	Backup and restore record	**Open**
G7	Officer provisioning and deprovisioning process defined and usable	Procedure and tooling	**Open**
G8	Decisions taken on BR-10, BR-11 and BR-12	Signed decisions from the municipality	**Open**
G9	Data-protection, hosting-location and retention requirements validated by counsel	Written validation	**Open**

Gate	Criterion	Evidence required	State
G10	Independent penetration test of the deployed system	Test report	**Not run**

9.2 Acceptance of the verification feature

- A badge for an active rider with a current licence returns valid; revoking the rider makes the next scan return flagged.
- An anonymous request for the same badge returns only a masked name and the verdict, and creates no log entry.
- An officer of the same municipality sees the full view and a log entry is created; an officer of another municipality does not.
- With the phone offline, a previously downloaded rider is answered from the snapshot, labelled with the snapshot date, and the scan appears in the enforcement log after reconnecting, exactly once.